Project Report

Name: Arpit Parmar

Reg No.: 25BAI11617

Branch: Computer Science (AI & ML)

Course: Fundamentals of AI & ML

Date: March 29, 2026

Tic Tac Toe AI using Minimax Algorithm

Introduction

This project was developed as part of the Fundamentals of Artificial Intelligence and Machine Learning course. The aim of the project is to demonstrate how intelligent decision-making can be implemented using algorithms in a simple game environment. Tic Tac Toe, though a basic game, provides an excellent platform to understand core AI concepts such as recursion, state evaluation, and optimal decision-making. In this project, I developed a complete system where a human player competes against an AI powered by the Minimax algorithm.

Project Overview

The project consists of two main implementations: a console-based version and a graphical user interface (GUI) version. The console version allows users to interact through terminal inputs, while the GUI version enhances usability through clickable buttons and a visual board. Both versions use the same AI logic to ensure consistency in gameplay.

Working of the System

The system represents the Tic Tac Toe board as a list of nine positions. Players take turns marking positions until either a player wins or the board is full. The AI uses the Minimax algorithm to evaluate all possible future game states. It recursively explores each possible move and assigns scores based on the outcome. The move with the best score is selected, ensuring that the AI always plays optimally.

Minimax Algorithm

The Minimax algorithm is a recursive decision-making algorithm used in game theory. It operates on the principle that one player tries to maximize their score while the opponent tries to minimize it. In this project, the AI acts as the maximizing player and the human acts as the minimizing player. The algorithm explores all possible moves until it reaches a terminal state (win, loss, or draw) and then propagates scores back up the recursion tree.

Implementation Details

The project is implemented using Python. The console version handles user input through standard input, while the GUI version is built using the Tkinter library. Functions are defined for checking winning conditions, handling user input, generating available moves, and executing the Minimax algorithm. The GUI version also includes additional features such as restart functionality and player selection.

Results and Observations

During testing, it was observed that the AI never loses if it plays optimally. In most cases, when the human player also makes optimal moves, the game results in a draw. The GUI version significantly improves user experience compared to the console version. This confirms the effectiveness of the Minimax algorithm in solving decision-making problems.

Learning Outcomes

Through this project, I gained a strong understanding of recursion and algorithm design. I also learned how AI can be applied to real-world problems, even in simple games. Additionally, I developed skills in Python programming and GUI development using Tkinter.

Future Scope

The project can be enhanced by implementing Alpha-Beta pruning to improve efficiency. It can also be extended to support multiplayer functionality and deployed as a web or mobile application. Adding difficulty levels and larger grids would further improve the system.

Conclusion

This project successfully demonstrates the application of artificial intelligence in game development. The Minimax algorithm ensures optimal gameplay and provides a strong foundation for understanding AI concepts. It serves as a stepping stone for more advanced AI-based systems.